### Listing 2.9 `activated` behavior from `WalletTimer`

```
import scala.concurrent.duration._

object WalletTimer {
   ...

  def activated(total: Int): Behavior[Command] =
    Behaviors.receive { (context, message) =>
      Behaviors.withTimers { timers =>                    ❶
        message match {
          case Increase(amount) =>
            val current = total + amount
            context.log.info(s"increasing to $current")
            resume(current)
          case Deactivate(t) =>
            timers.startSingleTimer(Activate, t.second)   ❷
            deactivated(total)
          case Activate =>
            Behaviors.same
        }
      }
   }
}
```

❶ Behavior factory creating timers

❷ Schedules Activate in t seconds. Converts t as Int to FiniteDuration in seconds.

The rest of the wallet can stay the same. With the message scheduling, you have added the new functionality. Pretty simple, isn't it?

# *Summary*

- The `ActorSystem` is the first actor. At the same time, it represents the entire system.